



Faculté des Sciences et Technologies
Master 2 Ingénierie de Systèmes Complexes (ISC)

TD/TP

Optimisation numérique

Alain Richard

Mise à jour : 26 septembre 2024

Contributeurs : Julien Flamant, Paul Catala, El-Hadi Djermoune

Table des matières

1	Introduction à l'optimisation	1
1.1	Questions de cours	1
1.2	Formulation d'un problème d'optimisation	1
1.3	Moindres carrés linéaires	2
1.4	Méthode de descente de gradient	2
1.5	Méthode de Newton	3
2	Optimisation sans contraintes : moindres carrés linéaires	5
2.1	Régression polynomiale	5
2.1.1	Construction du jeu de données	5
2.2	Moindres carrés linéaires	6
2.3	Sélection de l'ordre du polynôme	6
2.4	Résolution d'un problème inverse : déconvolution d'un signal triangulaire	7
2.4.1	Construction du jeu de données	7
2.4.2	Déconvolution	9
3	Optimisation sans contraintes : méthodes itératives	11
3.1	Régression non linéaire	11
3.1.1	Construction du jeu de données	11
3.1.2	Mise en œuvre de la méthode du simplexe de Nelder et Mead	12
3.1.3	Mise en œuvre d'une méthode de quasi-Newton : BFGS	13
3.1.4	Mise en œuvre d'une méthode de moindres carrés non linéaires : Levenberg-Marquardt	13

TD/TP 1

Introduction à l'optimisation

1.1 Questions de cours

1. On considère le problème $(\mathcal{P})$ d'optimisation suivant :

$$\min_{[x_1, x_2]^\top} (x_1 - 3)^2 + (x_2 + 2)^2 \quad \text{tel que} \quad \begin{cases} 3x_2^2 - x_1 = 0 \\ x_1 + x_2 \leq 2 \end{cases} \quad (\mathcal{P})$$

- (a) Donner le ou les variables du problème $(\mathcal{P})$.
 - (b) Donner l'expression de la fonction objectif du problème $(\mathcal{P})$.
 - (c) Donner les contraintes du problème $(\mathcal{P})$ et les classer par catégorie.
 - (d) À partir de votre analyse du problème $(\mathcal{P})$, proposer une simplification sous la forme d'un nouveau problème d'optimisation $(\mathcal{P}')$ équivalent.
2. On considère à présent le problème d'optimisation suivant

$$\min_{[x_1, x_2]^\top} 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

- (a) Caractériser le problème d'optimisation : variables, fonction objectif, contraintes.
 - (b) Soit $\mathbf{x}^* \in \mathbb{R}^2$. Énoncer une condition nécessaire pour que $\mathbf{x}^*$ soit un minimiseur local du problème d'optimisation.
 - (c) En déduire que $\mathbf{x}^* = [0, 0]^\top$ n'est pas un minimiseur local du problème.
3. Soit $f : \mathbb{R} \rightarrow \mathbb{R}$ une fonction. Donner la définition de la convexité pour f .
 4. Expliquer pourquoi la convexité est une propriété intéressante en optimisation numérique.

1.2 Formulation d'un problème d'optimisation

Des boulangeries livrent chaque matin des croissants dans des cafés pour les consommateurs. Il y a n boulangeries qui produisent chacune une quantité (fixe) $p_1, \dots, p_n$ de petits pains, et m cafés, qui en consomment une quantité (fixe) $q_1, \dots, q_m$, respectivement. La quantité totale produite est toujours égale à la quantité totale consommée. Chaque boulangerie peut livrer à plusieurs cafés. La livraison de la i -ème boulangerie au j -ème café a un coût c_{ij} . On souhaite déterminer le nombre de petits pains que chaque boulangerie doit expédier à chaque café de manière à minimiser le coût total des livraisons.

1. Quelles sont les variables de ce problème d'optimisation ?
2. Exprimer la fonction objectif de ce problème en fonction de l'ensemble des coûts c_{ij} .
3. Quelles sont les contraintes du problème ? Ecrire le problème d'optimisation final.

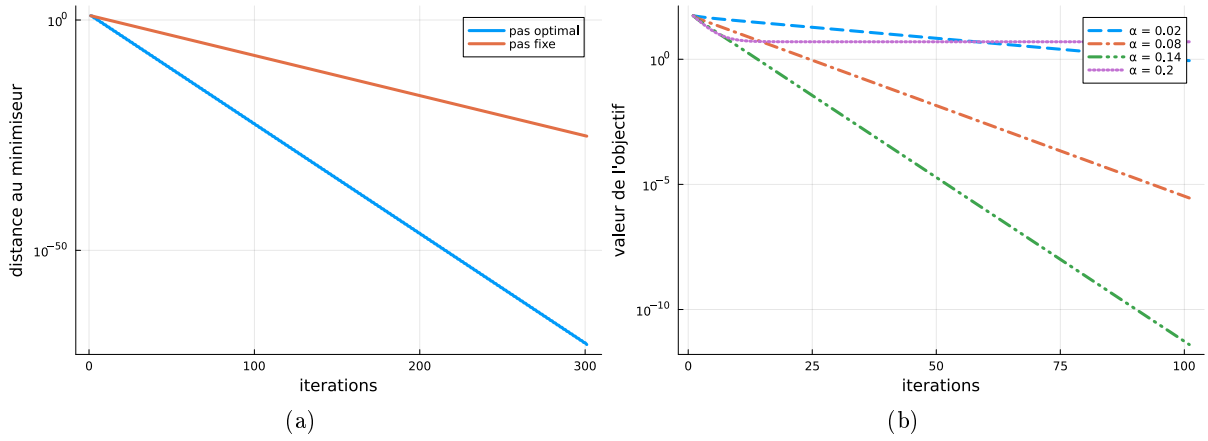


FIGURE 1.1 – Évolution de : (a) $\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_2^2$; (b) la fonction objectif f , en fonction des itérations

1.3 Moindres carrés linéaires

Soient x_1 et x_2 deux variables inconnues. On effectue trois mesures y_1, y_2 et y_3 reliées aux inconnues par le modèle suivant

$$\begin{cases} y_1 = 2x_1 + 3x_2 + b_1 \\ y_2 = 3x_2 + b_2 \\ y_3 = x_1 + b_3 \end{cases} \quad (1.1)$$

où $\mathbf{b} = [b_1, b_2, b_3]^\top$ est un vecteur inconnu modélisant l'erreur (dû à du bruit de mesure, à des erreurs de modèle, etc.)

1. Mettre le modèle (1.1) sous la forme générale $\mathbf{y} = \mathbf{A}\mathbf{x} + \mathbf{b}$, en explicitant et nommant les quantités $\mathbf{y}$, $\mathbf{A}$ et $\mathbf{x}$.
2. Afin d'estimer $\mathbf{x}$ à partir de $\mathbf{y}$, on cherche à présent à minimiser la norme euclidienne du vecteur d'erreur $\mathbf{b}$. En déduire le problème d'optimisation correspondant, en fonction de $\mathbf{y}$, $\mathbf{A}$ et $\mathbf{x}$.
3. En détaillant les calculs, écrire le problème d'optimisation sous la forme standard

$$\min_{\mathbf{x}} \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{p}^\top \mathbf{x}$$

où l'on donnera l'expression de la matrice $\mathbf{Q}$ et du vecteur $\mathbf{p}$ en fonction de $\mathbf{A}$ et $\mathbf{y}$.

4. Calculer le gradient de la fonction objectif en fonction de $\mathbf{Q}$ et $\mathbf{p}$.
5. Donner l'expression matricielle du minimiseur (global) du problème en fonction de $\mathbf{Q}$ et $\mathbf{p}$, puis en fonction de $\mathbf{A}$ et $\mathbf{y}$.

1.4 Méthode de descente de gradient

1. On considère la fonction objectif $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ suivante

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x}, \text{ avec } \mathbf{Q} = \begin{bmatrix} 1 & 0 \\ 0 & \gamma \end{bmatrix}, \quad \gamma > 0$$

- (a) Calculer $\nabla f(\mathbf{x})$.
- (b) On cherche à minimiser f par une stratégie de descente de gradient. Donner l'expression d'une itération k de cet algorithme en fonction du pas α_k .

- (c) Donner l'expression de $\mathbf{x}_1$ en fonction de α_0 pour $\mathbf{x}_0 = [\gamma, 1]^\top$.
 - (d) Trouver le pas optimal pour cette itération.
2. Les figures ci-dessus montrent les résultats d'algorithmes de descente de gradient appliqués à la fonction f .
- (a) La figure 1.1(a) présente l'évolution de la distance $\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_2^2$ où $\mathbf{x}^*$ est le minimiseur global de f pour deux stratégies de choix de pas : pas constant et pas optimal. Associer chaque courbe à la stratégie correspondante. Justifier votre réponse.
 - (b) La figure 1.1(b) présente la valeur de la fonction f au cours des itérations, dans une stratégie de pas fixe, pour différentes valeurs de pas. Commenter la figure et en déduire des recommandations pour le choix du pas de descente. Que se passe-t-il si l'on choisit un pas fixe $\alpha = 1$?

1.5 Méthode de Newton

On considère la fonction $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ définie par

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} - \mathbf{b}^\top \mathbf{x}, \text{ avec } \mathbf{Q} \succ 0$$

1. Calculer $\nabla f(\mathbf{x})$ et en déduire son minimiseur
2. Calculer $\nabla^2 f(\mathbf{x})$
3. Exprimer $\mathbf{x}_{k+1}$ en fonction de $\mathbf{x}_k$ pour une itération de Newton à pas constant. Qu'en déduit-on ?

TD/TP 2

Optimisation sans contraintes : moindres carrés linéaires

2.1 Régression polynomiale

L'objectif de cet exercice est d'appréhender la méthode des moindres carrés linéaires pour estimer les paramètres d'un modèle polynomial à partir de données expérimentales.

2.1.1 Construction du jeu de données

Pour évaluer les performances des méthodes numériques, il est d'usage de simuler des jeux de données *via* un modèle dont on connaît la « vraie »¹ valeur des paramètres. Cette « vraie » valeur du vecteur des paramètres n'est bien sûr pas utilisée dans la procédure de résolution du problème posé.

1. Pour $\mathbf{x}=[0:0.5:15]'$; de dimension $N \times 1$, simuler le modèle polynomial

$$\mathbf{y} = h_0 + h_1\mathbf{x} + h_2\mathbf{x}^2 + h_3\mathbf{x}^3 + h_4\mathbf{x}^4$$

pour un vecteur des paramètres vrais, de dimension $p \times 1$, donné par

$$\mathbf{h}=[500 \ 10 \ -10 \ -5 \ 0.4]';$$

Représenter $\mathbf{y}(\mathbf{x})$.

2. Pour s'approcher de la réalité, les incertitudes (souvent qualifiées de bruit) sur les mesures sont simulées par une suite stationnaire² de N variables aléatoires distribuées selon une loi normale $\mathcal{N}(\mu, \sigma^2)$ de moyenne $\mu = 0$ et de variance $\sigma^2 = 10^2$: $\mathbf{b}=\mu+\sigma.*\text{randn}(N,1);$. Ces incertitudes $\mathbf{b}$ sont supposées additives et les observations disponibles sont notées

$$\mathbf{y}_b = \mathbf{y} + \mathbf{b}.$$

3. Représenter les signaux $\mathbf{y}_b(x)$ et $\mathbf{b}(x)$ et relancer plusieurs fois le code. Que constatez-vous ?
4. Calculer l'énergie du bruit W_b puis modifier la variance de celui-ci. Que constatez-vous ?

1. La notion de valeur « vraie » des paramètres n'a aucun sens en pratique car cela supposerait qu'il y a un « vrai » modèle; or un modèle mathématique, aussi fin soit-il, n'est qu'une approximation de la réalité qui est trop complexe pour être parfaitement modélisée.

2. L'hypothèse de stationnarité traduit le fait que l'on suppose que moyenne et variance des incertitudes ne varient pas au cours du temps. Il s'agit ici d'une suite variables aléatoires indépendantes et identiquement distribuées; cette terminologie est souvent abrégé par i.i.d. Lorsque la moyenne est nulle, l'indépendance des variables aléatoires confère à la densité spectrale du bruit une propriété particulière et l'on parle alors de bruit blanc.

2.2 Moindres carrés linéaires

1. Construire la matrice de Vandermonde $\mathbf{X}$ à partir du vecteur $\mathbf{x}$.
2. Déterminer le vecteur $\hat{\mathbf{h}}$ en utilisant la formulation matricielle des moindres carrés et comparer le résultat obtenu avec la valeur « vraie » des paramètres. L'instruction `inv` permet d'inverser une matrice.
3. Dans le cas général, il n'est pas recommandé d'utiliser la programmation *via* `inv` qui est coûteuse en termes de calcul et peut souffrir de problèmes de conditionnement. Plusieurs alternatives sont alors possibles :

```
% *****
% moindres carrés \emph{via} élimination gaussienne
h_est2=(X'*X)\X'*y_b

% *****
% moindres carrés \emph{via} factorisation QR
[Q,R] = qr(X);
QTY = Q'*y_b;
opts_UT.UT = true;
h_est3 = linsolve(R,QTY,opts_UT)
Cond_QR = cond(R)

% *****
% moindres carrés \emph{via} SVD
[U,S,V] = svd(X,'econ');
h_est4 = V*inv(S)*U'*y_b
Cond_SVD = cond(S)

% *****
% moindres carrés \emph{via} pseudo-inverse (obtenue après SVD)
h_est5=pinv(X)*y_b
```

Tester ces différentes alternatives. En principe, les résultats devraient être identiques.

Dans le cas général, il convient d'utiliser les approches fondées sur une factorisation QR ou sur une décomposition en valeurs singulières SVD. Si la SVD est plus complexe que la décomposition QR elle fournit le conditionnement du problème et permet la régularisation de problèmes mal conditionnés [?].

4. Afficher la matrice $\mathbf{S}$ des valeurs singulières et vérifier que le rapport des valeurs singulières extrêmes donne le conditionnement au sens de la norme 2.
5. Calculer et représenter la sortie estimée $\hat{\mathbf{y}}(x)$ ainsi que les résidus $\boldsymbol{\epsilon}(x) = \mathbf{y}_b(x) - \hat{\mathbf{y}}(x)$.
6. Que vaut le minimum du critère d'optimisation ? Comparer cette valeur à l'énergie du bruit additif calculée précédemment W_b .
7. Tester également le cas où il n'y a pas de bruit sur les données.

2.3 Sélection de l'ordre du polynôme

Lorsque l'on cherche à ajuster un polynôme de degré d à des données expérimentales, l'ordre de ce polynôme est généralement inconnu. Se pose alors le problème de sélection de l'ordre le plus approprié qui peut se formuler comme un problème d'optimisation discrète : rechercher le meilleur ordre au sens d'un certain critère.

1. Considérons dans un premier temps que le critère retenu soit celui des moindres carrés. Calculer les valeurs de ce critère pour d variant de 1 à 10. Que constatez-vous ? À partir de quelle valeur de d ce critère pourra-t-il être nul ?

2. Afin de palier l'inconvénient illustré à la question précédente, les techniques de sélection d'ordre s'appuient sur des critères qui pénalisent le nombre de paramètres p afin de privilégier des modèles parcimonieux (pour cet exercice $p = d + 1$). En effet, l'incertitude d'estimation qui pèse sur chaque paramètre s'accroît avec le nombre de paramètres (l'information disponible *via* les données, qui est fixe, se répartit sur un plus grand nombre d'entre eux).

Le critère d'information d'Akaike (AIC) permet de réaliser cette sélection d'ordre en retenant celui qui minimise

$$AIC = 2 \ln(\|\epsilon\|_2^2) + 2p.$$

Déterminer l'ordre du polynôme qui minimise le critère d'information d'Akaike.

2.4 Résolution d'un problème inverse : déconvolution d'un signal triangulaire

Objectif de l'exercice

Le problème considéré est celui de la reconstruction d'un signal x , supposé inconnu car non accessible à une mesure directe, ayant subi un filtrage gaussien inhérent à l'instrument de mesure. Il s'agit d'un filtrage linéaire d'usage courant en traitement d'image ; il est utilisé, ici, dans un contexte monodimensionnel. La réponse impulsionnelle (RI) d'un filtre gaussien est définie par :

$$h(t) = \frac{1}{\sigma_h \sqrt{2\pi}} \exp\left(-\frac{(t - m_h)^2}{2\sigma_h^2}\right). \quad (2.1)$$

Le signal y mesuré en sortie du filtre, seul accessible, est supposé être corrompu par des incertitudes additives. En effet, les mesures en sortie de filtre sont à minima sujettes au bruit de quantification dû au convertisseur analogique-numérique de la chaîne de mesure.

Il s'agit d'appréhender le caractère instable de l'opération de déconvolution, lorsque celle-ci est effectuée de façon « naïve », soit par division polynomiale (fonction `deconv` de Matlab), soit par filtrage inverse (dans le domaine fréquentiel ou par moindres carrés qui est l'équivalent matriciel). Il est fortement inspiré par l'exemple donné en introduction de l'ouvrage collectif [?]. En présence d'erreurs (de bruit) sur les données, même très faibles, les résultats fournis par ces méthodes d'inversion peuvent, en effet, être inexploitable. Il s'agit là d'un problème inverse *mal posé*. La réponse en fréquence du système à inverser permet de caractériser le degré de difficulté du problème. L'exercice illustrera la (les) solution(s) obtenues par régularisation linéaire quadratique.

2.4.1 Construction du jeu de données

La fréquence d'échantillonnage est fixée à la valeur $f_e = 1$ Hz. Les échantillons du signal d'entrée sont notés $x[n]$, ceux de la réponse impulsionnelle du filtre $h(n)$ et ceux du signal mesuré en sortie $y(n)$. Le signal mesuré en sortie est sujet à des incertitudes additives notées $b(n)$ et le problème direct peut alors être modélisé comme suit :

$$y(n) = (x \star h)(n) + b(n). \quad (2.2)$$

Simulation du signal d'entrée

Le signal d'entrée, supposé être triangulaire, est défini par :

$$x(t) = \begin{cases} \frac{t}{L_x/2}, & \forall t \in [0, \frac{L_x}{2}], \\ \frac{-t+L_x}{L_x/2}, & \forall t \in [\frac{L_x}{2}, L_x]. \end{cases}$$

Simuler le signal à temps discret $x(n)$ correspondant, pour une durée d'observation $L_x = 100$ s.

Aide : remplacer t par nT_e dans l'équation définissant $x(t)$. Il est possible d'utiliser une instruction du type, $\mathbf{x}=[\mathbf{x1} \ \mathbf{x2}]$, où $\mathbf{x1}$ et $\mathbf{x2}$ représentent deux tronçons successifs du signal. De manière générale, il est conseillé d'adapter le script ci-dessous qui permet de maîtriser les différents paramètres de la discrétisation.

```
Lx = 100;           % durée d'observation du signal d'entrée (secondes)
fe = 1;            % fréquence d'échantillonnage en (Hz)
Te = 1/fe;         % période d'échantillonnage (secondes)
Nx = Lx/Te;        % nombre d'échantillons du signal x
nx = 0:Nx-1;       % indice temporel (entier)
tx= nx*Te;         % base de temps (secondes)
```

Discrétisation de la réponse impulsionnelle

Pour les besoins de la simulation, déterminer la réponse impulsionnelle $h(n)$ du filtre gaussien obtenue par discrétisation de l'équation (2.1). Le filtre est de plus supposé causal et ses paramètres seront pris égaux respectivement à $m_h = 15$ et $\sigma_h = 4$. Enfin, la réponse impulsionnelle est tronquée à un horizon de 30 s. Représenter cette réponse impulsionnelle.

Rappel : la réponse impulsionnelle d'un filtre causal est nulle aux temps négatifs.

Simulation du problème direct, première approche

1. Comme le filtre gaussien est un filtre **linéaire invariant**, il est possible de calculer sa sortie par le produit de convolution discret donné par l'équation (2.2). Simuler le signal de sortie non bruité $y - nb(n)$ à l'aide de la fonction `conv`.
2. Si le signal de sortie y est mesuré à l'aide d'un convertisseur analogique-numérique, il est sujet au bruit de quantification. L'opération de quantification peut être simulée par l'instruction `y=round(ynb*2^10)/2^10`. Simuler le signal quantifié y et représenter celui-ci à l'aide de la fonction `stairs`.
3. Déterminer et représenter le bruit de quantification $b(n)$.

Simulation du problème direct par approche matricielle

Dans cette partie, le signal de sortie non bruité est simulé par l'équation matricielle $\mathbf{y}_{nb} = \mathbf{H}\mathbf{x}$. L'usage d'un paramétrage soigné est recommandé pour permettre de modifier aisément les conditions de simulation et d'affichage des résultats.

1. Déterminer la dimension N_y du vecteur de sortie $\mathbf{y}_{nb}$ à partir des dimensions N_x et N_h des vecteurs d'entrée $\mathbf{x}$ et de réponse impulsionnelle $\mathbf{h}$ afin d'obtenir le même nombre d'échantillons que par la fonction `conv`.
2. Construire la matrice de convolution $\mathbf{H}$ à l'aide de la fonction `toeplitz`. Pour cela, il est proposé d'initialiser le premier vecteur ligne et le premier vecteur colonne de $\mathbf{H}$ avec des vecteurs nuls de dimension appropriée à l'aide la fonction `zeros`, puis de remplacer certains de leurs éléments avec les éléments de $\mathbf{h}$.
3. Déterminer la sortie non bruitée par une opération matricielle, puis appliquer la fonction permettant de simuler l'erreur de troncature.

2.4.2 Déconvolution

Déconvolution « naïve » et constat de l'instabilité

1. Reconstruire le signal d'entrée en utilisant l'opération de déconvolution de Matlab (`deconv`), en se plaçant, dans un premier temps, dans le cas idéal où l'on a accès à la sortie non bruitée y_{nb} .
2. À présent, reconstruire le signal d'entrée à partir des échantillons mesurés du signal de sortie y (cadre réaliste). Le signal reconstruit sera appelé x_{rec} . Que vaut le RSB du signal de sortie ?
3. Étudier de façon empirique l'influence de la « forme » de la réponse impulsionnelle sur la qualité de la reconstruction en modifiant la valeur du paramètre σ_h .

Interprétation fréquentielle de l'inversion

1. Déterminer la transformée de Fourier discrète (TFD) H du filtre. Tracer sa réponse en fréquence (en dB).
2. Déterminer les TFD X du signal d'entrée et du signal de sortie Y . Tracer les densités spectrales de puissance correspondantes.
3. Comment aurait-il été possible de réaliser l'opération de convolution dans le domaine fréquentiel ?
4. En déduire une seconde formulation de l'opération de déconvolution, par filtrage inverse, dans le domaine fréquentiel. Autrement dit, trouver la TFD X_{rec2} à partir des TFD H et Y puis la représentation temporelle x_{rec2} . Programmer ces opérations et représenter le signal ainsi reconstruit (à partir des échantillons bruités et non bruités de la sortie).
5. Pour analyser les piètres performances de l'inversion « naïve » il est proposé de représenter :
 - la densité spectrale de puissance $S_w(f)$ du bruit de quantification ;
 - la DSP $S_y(f)$ du signal de sortie ;
 - la densité spectrale d'énergie du filtre inverse.
6. En déduire l'interprétation fréquentielle du problème posé par l'opération de déconvolution. Il peut, à nouveau, être intéressant de faire varier le paramètre σ_h .

Déconvolution par moindres carrés (l_2)

1. Calculer le signal déconvolué à l'aide de l'estimateur des moindres carrés.
2. Tracer le signal déconvolué et calculer l'énergie de l'erreur de reconstruction.
3. Tracer le spectre des valeurs singulières de la matrice $\mathbf{H}^\top \mathbf{H}$ et déterminer son conditionnement.
4. Comment le conditionnement évolue-t-il en fonction de la valeur du paramètre σ_h du filtre ?

Déconvolution par régularisation linéaire quadratique (l_2l_2)

Cette partie concerne la mise en œuvre de la solution obtenue par minimisation d'un critère composite de fonctionnelles $\mathcal{F}_b$ et $\mathcal{F}_x$ quadratiques : $J(x) = \mathcal{F}_b\{\mathbf{x}\} + \lambda \mathcal{F}_x\{\mathbf{x}\}$

1. Construire la matrice carrée $\mathbf{D}_1$, de dimension N_x , telle que le produit $\mathbf{D}_1 \mathbf{x}$ donne les différences premières³ : $x(n+1) - x(n)$.

Aide :

3. Les différences premières, lorsqu'elles sont divisées par le pas T_e , correspondent à une approximation de la dérivée première.

```

dcol1 = zeros(1,Nx);
dcol1(1:2) = dcol1(1:2) + [1 -1];
dlig1 = zeros(1,Nx);
dlig1(1,1) = dcol1(1,1);
D1 = toeplitz(dcol1,dlig1);

```

Variante : le remplacement de la seconde ligne du script ci-dessus par

```

dcol1(1:3) = dcol1(1:3) + [1 -2 1];

```

donne accès aux différences secondes définissant alors la matrice $\mathbf{D}_2$.

2. Calculer la solution régularisée en déterminant le coefficient de régularisation ? de manière empirique⁴.
3. En simulation, le signal à reconstruire est parfaitement connu, il est donc possible de déterminer la valeur du coefficient de régularisation qui minimise l'énergie de l'erreur de reconstruction. Tracer l'évolution de l'énergie de l'erreur de reconstruction en fonction de λ . Ecrire une procédure permettant d'approcher la valeur optimale de λ , par recherche exhaustive sur un intervalle, à l'aide de la fonction `min`.
4. Tracer la « courbe en L » en représentant, sur une échelle log-log, la fonctionnelle de régularisation $\mathcal{F}_x\{\hat{\mathbf{x}}(\lambda)\} = \|\mathbf{D}_1\hat{\mathbf{x}}(\lambda)\|_2^2$ en fonction du critère d'adéquation aux données $\mathcal{F}_b\{\hat{\mathbf{x}}(\lambda)\} = \|\mathbf{y} - \mathbf{H}\hat{\mathbf{x}}(\lambda)\|_2^2$ (moindres carrés).

Aide : vous pouvez faire varier λ de la manière suivante :

```

% Intervalle de variation du coefficient de régularisation
minlambda=-7;
paslambda=0.1;
maxlambda=+2;
ilambda=0;
for varlambda=minlambda:paslambda:maxlambda,
    lambda=10^varlambda;
    ilambda=ilambda+1;
    ..

```

5. Que se passe-t-il lorsque l'on remplace la matrice $\mathbf{D}_1$ par la matrice identité $\mathbf{I}_{N_x}$ ou par $\mathbf{D}_2$? Quelle pénalisation constitue une façon appropriée de coder les connaissances *a priori* dont on dispose sur le signal à reconstruire.

4. Par exemple, en testant plusieurs valeurs dans un intervalle conséquent, comme $[10^{-7}, 10^2]$.

TD/TP 3

Optimisation sans contraintes : méthodes itératives

3.1 Régression non linéaire - test de différents algorithmes itératifs d'estimation des paramètres

L'objectif de cet exercice est d'éprouver différents algorithmes itératifs d'optimisation pour estimer, à partir de données expérimentales, les paramètres d'un modèle non linéaire en ces paramètres. En l'occurrence, le modèle est une somme d'exponentielles et comporte trois paramètres [?].

3.1.1 Construction du jeu de données

1. Pour $\mathbf{x}=[0:0.1:7]'$, de dimension $N \times 1$, simuler le modèle non linéaire en les paramètres donné par l'expression suivante

$$\mathbf{y}_{nb} = h_1(e^{-h_2\mathbf{x}} - e^{-h_3\mathbf{x}})$$

pour un vecteur des paramètres vrais, de dimension $p \times 1$, donné par

$$\mathbf{h}=[10 \ 1 \ 2]';$$

Représenter $\mathbf{y}_{nb}(\mathbf{x})$ à l'aide de l'instruction

```
plot(x,y,'o','MarkerEdgeColor','g','MarkerSize',7)
```

Pour la suite de l'exercice il sera intéressant de programmer le modèle sous forme de fonction Matlab (à placer en fin de script) :

```
function [ymod]= somme_exp(h,x)
ymod = h(1)*(exp(-h(2)*x)-exp(-h(3)*x));
end
```

En effet, les algorithmes d'optimisation vont avoir besoin de calculer la sortie de ce modèle pour plusieurs jeux de paramètres (jusqu'à trouver le jeu de paramètres optimal). C'est l'étape de simulation propre à tout algorithme d'optimisation itératif.

2. Pour s'approcher de la réalité, les incertitudes sur les mesures sont simulées par une suite stationnaire de N variables aléatoires distribuées selon une loi normale $\mathcal{N}(\mu, \sigma^2)$ de moyenne $\mu = 0$ et de variance $\sigma^2 = 0.22$. Ces incertitudes $\mathbf{b}$ sont supposées additives et les observations disponibles sont notées

$$\mathbf{y} = \mathbf{y}_{nb} + \mathbf{b}$$

3.1.2 Mise en œuvre de la méthode du simplexe de Nelder et Mead

La méthode du simplexe de Nelder et Mead est programmée au sein de la librairie de Matlab dédiée à l'optimisation : `Optimization Toolbox`. Elle est appelée *via* la fonction `fminsearch` dont il est possible de consulter l'aide en ligne (`help fminsearch`) et la documentation (`doc fminsearch`). Cette méthode peut être mise en œuvre par les deux lignes de script suivantes :

```
[hmin,Jmin] = fminsearch(@(h) critere(y,h,x),h0)
```

Elle nécessite préalablement la définition de la fonction dénommée `critere(y,h,x)` ci-dessus.

1. Dans un premier temps, il est proposé d'utiliser comme critère, le carré de la norme l_2 de l'écart entre le vecteur de mesure $\mathbf{y}$ et le vecteur de sortie du modèle $\mathbf{y}_{mod}$:

$$J_2 = \sum_{n=1}^N (y(n) - y_{mod}(n))^2 = \|\mathbf{y} - \mathbf{y}_{mod}\|_2^2 = (\mathbf{y} - \mathbf{y}_{mod})^\top (\mathbf{y} - \mathbf{y}_{mod})$$

Programmer une seconde fonction Matlab qui assurera le calcul du critère l_2 . Elle devra, comme la fonction correspondant au modèle, être placée en fin de script.

2. Tester la méthode du simplexe de Nelder et Mead
 - en initialisant la recherche avec la valeur `h0=[1 1 1]'` ;
 - et en considérant que les données mesurées $\mathbf{y}$ sont non bruitées (il suffit de régler $\sigma = 0$). Cette initialisation ainsi que les données non bruitées seront également utilisées pour les questions suivantes.
3. Enrichir la récupération d'informations sur le déroulement de l'algorithme
 - (a) en utilisant le script suivant :

```
[hmin,Jmin,exitflag,output] = fminsearch(@(h) critere(y,h,x),h0)
```

Noter le nombre d'itérations.

- (b) puis en complétant par la ligne :

```
options = optimset('Display','iter','TolFun',1.e-4,'TolX',1.e-4)
[hmin,Jmin,exitflag,output]=fminsearch(@(h) critere(y,h,x), ...
    h0,options)
```

Noter l'évolution du critère au cours des itérations et la variété des opérations utilisées (réflexion, expansion, contraction externe, contraction interne, rétrécissement). Consulter également la documentation relative à `optimset`.

4. **Choix du critère** : il est proposé de construire une seconde fonction de coût, correspondant à la norme l_1 de l'écart¹

$$J_1 = \sum_{n=1}^N |y(n) - y_{mod}(n)| = \|\mathbf{y} - \mathbf{y}_{mod}\|_1$$

Comparer les résultats obtenus (minimiseurs, nombres d'itérations, ...), pour le même jeu de données, entre une minimisation de la norme l_2 et la norme l_1 .

5. **Initialisation** : l'objectif à présent est de tester l'influence de l'initialisation. Essayer par exemple `h0=[0 0 0]'` et `h0=[100 100 100]'` ou tout autre triplet de votre choix. Pour ces cas, il est intéressant de modifier le paramètre optionnel `TolX` à la valeur `1e-8`. Que constatez-vous ?

Cela montre l'intérêt des recherches de minimiseurs dites *multistart* qui vont essayer plusieurs points initiaux tirés au hasard dans le domaine d'intérêt. Bien que cette stratégie puisse parfois trouver toutes les solutions, il n'y a aucune garantie qu'elle y parvienne.

1. La norme l_1 est non différentiable en raison de sa singularité en 0, mais l'algorithme de Nelder et Mead ne nécessite justement pas cette propriété.

6. **Données bruitées** : pour terminer, tester l'algorithme en présence de données bruitées. Il est intéressant de superposer sur le même tracé, les données, les sorties du modèle « vrai » ainsi que les sorties des modèles correspondant aux minimisations l_1 et l_2 .

3.1.3 Mise en œuvre d'une méthode de quasi-Newton : BFGS

La fonction `fminunc` (*find minimum of unconstrained multivariable function*) permet de mettre en œuvre différents algorithmes d'optimisation sans contrainte [?]. Elle dispose d'un paramètre guidant leur choix, selon la taille de l'espace des variables de décision :

- lorsque celles-ci sont en grand nombre (choix par défaut de l'algorithme), la fonction s'appuie sur une méthode à régions de confiance² (non étudiée en cours) ;
- lorsque celles-ci sont en nombre raisonnable (l'option `'LargeScale'` est mise en position `'off'`) plusieurs algorithmes sont utilisables :
 - par défaut : quasi-Newton fondé sur l'approximation de la matrice hessienne (`'bfgs'` Broyden-Fletcher-Goldfarb-Shanno associé à une recherche linéaire par interpolation cubique) ;
 - quasi-Newton fondé sur l'approximation de l'inverse de la matrice hessienne (`'dfp'` Davidon-Fletcher-Powell) ;
 - gradient (`'steepdesc'`) qui n'est pas très efficace dans le cas général.

Le choix de l'algorithme est alors effectué *via* les arguments optionnels :

```
Options=optimset('LargeScale','off','Display','iter', ...
    'TolX',1e-8,'TolFun',1e-10,'HessUpdate','bfgs');
```

1. Tester la méthode de quasi-Newton BFGS en activant la fonction `fminunc` de manière à récupérer les informations permettant de suivre l'évolution des itérations successives ainsi que la valeur du gradient et de la matrice hessienne à l'arrêt des itérations.
2. Comme précédemment, explorer le comportement de l'algorithme lorsque l'on éloigne le vecteur initial $\mathbf{h}_0$ de la solution parfaite (supposée inconnue) puis en présence de bruit sur les données.

3.1.4 Mise en œuvre d'une méthode de moindres carrés non linéaires (en les paramètres) : Levenberg-Marquardt

La méthode de Levenberg et Marquardt est mise en œuvre par la fonction `lsqnonlin`. Alors que les méthodes précédentes s'appuyaient directement sur le critère, passé comme argument de la fonction d'optimisation, il faut ici définir une nouvelle fonction calculant chacun des résidus. Les méthodes de moindres carrés non linéaires s'appuient automatiquement sur un coût l_2 de ces résidus (il n'est pas possible de choisir d'autres coûts, ce qui explique l'appellation de cette famille de méthodes).

1. Définir la fonction permettant de calculer les résidus.
2. Mettre en œuvre l'algorithme de Levenberg-Marquardt à l'aide des instructions suivantes :

```
optLM=optimset('Display','iter','Algorithm','levenberg-marquardt')
% il faut fournir des bornes inf. et sup. sur les variables de
% décision pour ne pas déclencher un message d'erreur, bien que
% ces bornes ne soient pas utilisées.
bornemin = zeros(3,1);
```

2. Les méthodes à régions de confiance (trust-region methods) utilisent un modèle quadratique comme approximation de la fonction d'objectif sur une région de confiance pour choisir simultanément la direction et le pas du déplacement du vecteur de décision. La taille de la région de confiance est mise à jour sur la base des performances passées de l'algorithme.

```
bornemin(:) = -Inf;  
bornemax = zeros(3,1);  
bornemax(:) = Inf;  
[hminLM,JminLM] = lsqnonlin(@(h) residu(y,h,x),h0,bornemin, ...  
    bornemax,optLM)
```

3. Comme précédemment, tester cet algorithme sous différentes conditions.
4. En conclusion, dresser un comparatif et une analyse critique (forces, faiblesses) des différents algorithmes testés.